Modul 2
Matematika Diskrit Graf
Properti, Representasi, dan Isomorfisme Graf





105 – Ivanka Bunga Aulia– 26/10/2026

123 –Syadida Tsaqifa Nada– 26/10/2026

124 – Zahra Azkiya Rahmani – 26/10/2026

Contents

Tujuan Pembelajaran 3
Materi Esensial 3
Tutorial Praktikum 4
Task Praktikum 5
Task Individu 14
Task01_Individu.py 14
Task02_Individu.py 14
Task Kelompok (Soal Kolaboratif & Sharing) 15
Task03_Timproyek.py 15
Task04_FullMesh.py 16
Task05_Datasister.py 17

Tujuan Pembelajaran

- Mahasiswa mampu mengidentifikasi dan membuat graf-graf khusus (*Complete, Cycle, Wheel, Bipartite, Directed*).
- Mahasiswa mampu menganalisis properti dasar graf (derajat, simpul terisolasi, simpul bandul) dan memverifikasi *Handshaking Theorem*.
- Mahasiswa mampu membuat berbagai representasi graf (*Adjacency List*, *Adjacency Matrix*, *Incidence Matrix*).
- Mahasiswa mampu memeriksa *isomorphism* dan menerapkannya pada studi kasus nyata.

Materi Esensial

Graf Khusus:

- **Complete Graph (K_n)**: Setiap simpul terhubung ke setiap simpul lain.
- **Cycle Graph (C_n)**: Simpul-simpul membentuk siklus tertutup.
- **Wheel Graph (W_n)**: C_n dengan satu simpul pusat yang terhubung ke semua simpul.
- **Bipartite Graph**: Simpul dibagi dua himpunan, sisi hanya antar himpunan.

Properti Dasar:

- **Degree (**deg(v)**)**: Jumlah sisi yang insiden dengan simpul v.
- **In-degree (**$\deg^-(v)$**)** & **Out-degree (**$\deg^+(v)$**)**: Hanya untuk *directed graph*.
- Handshaking Theorem: $\sum v \in V\ deg(v) = 2|E|$ (untuk tak berarah) dan $\sum deg-(v) = \sum deg+(v) = |E|$ (untuk berarah).
- **Isolated Vertex**: $deg(v) = 0$.
- **Pendant Vertex**: $deg(v) = 1$.

Representasi Graf:

- **Adjacency List**: Kamus (*dictionary*) di mana kunci adalah simpul dan nilai adalah daftar tetangganya.
- **Adjacency Matrix**: Matriks $N \times N$, entri (i, j) bernilai 1 jika ada sisi dari i ke j, 0 jika tidak.
- **Incidence Matrix**: Matriks $N \times M$, entri (i, j) bernilai 1 jika simpul i adalah ujung dari sisi j, 0 jika tidak.

Isomorfisme:

- Dua graf isomorfik jika mereka memiliki struktur yang sama.
- Untuk membuktikan dua graf **tidak** isomorfik, cari perbedaan pada invarian-invarian graf (jumlah simpul, jumlah sisi, deretan derajat).

Materi untuk Praktikum

Graph Khusus

EXAMPLE 5 **Complete Graphs** A **complete graph on n vertices**, denoted by K_n, is a simple graph that contains exactly one edge between each pair of distinct vertices. The graphs K_n, for $n = 1, 2, 3, 4, 5, 6$, are displayed in Figure 3. A simple graph for which there is at least one pair of distinct vertex not connected by an edge is called **noncomplete**. ◀

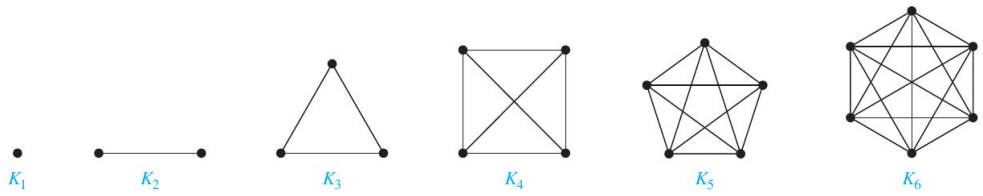


FIGURE 3 **The graphs K_n for $1 \leq n \leq 6$.**

EXAMPLE 6 **Cycles** A **cycle C_n**, $n \geq 3$, consists of n vertices $v_1, v_2, \ldots, v_n$ and edges $\{v_1, v_2\}, \{v_2, v_3\}, \ldots, \{v_{n-1}, v_n\}$, and $\{v_n, v_1\}$. The cycles C_3, C_4, C_5, and C_6 are displayed in Figure 4. ◀

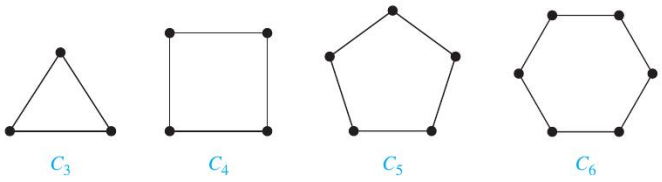


FIGURE 4 **The cycles C_3, C_4, C_5, and C_6.**

EXAMPLE 7 **Wheels** We obtain a **wheel W_n** when we add an additional vertex to a cycle C_n, for $n \geq 3$, and connect this new vertex to each of the n vertices in C_n, by new edges. The wheels W_3, W_4, W_5, and W_6 are displayed in Figure 5. ◀

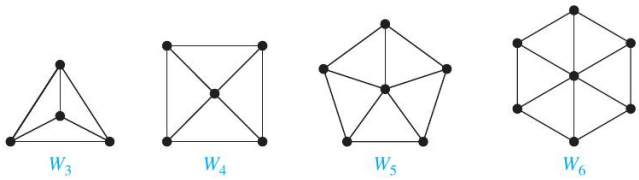


FIGURE 5 **The wheels W_3, W_4, W_5, and W_6.**

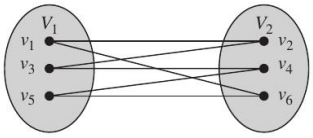


FIGURE 7 **Showing that C_6 is bipartite.**

EXAMPLE 9 C_6 is bipartite, as shown in Figure 7, because its vertex set can be partitioned into the two sets $V_1 = \{v_1, v_3, v_5\}$ and $V_2 = \{v_2, v_4, v_6\}$, and every edge of C_6 connects a vertex in V_1 and a vertex in V_2. ◀

EXAMPLE 13 **Complete Bipartite Graphs** A **complete bipartite graph $K_{m,n}$** is a graph that has its vertex set partitioned into two subsets of m and n vertices, respectively with an edge between two vertices if and only if one vertex is in the first subset and the other vertex is in the second subset. The complete bipartite graphs $K_{2,3}$, $K_{3,3}$, $K_{3,5}$, and $K_{2,6}$ are displayed in Figure 9. ◀

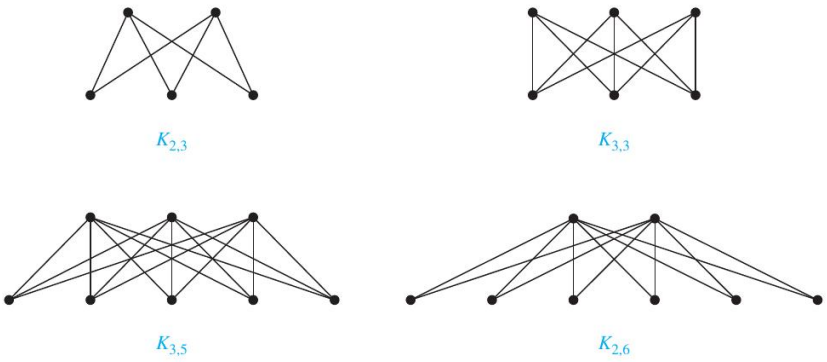


FIGURE 9 **Some complete bipartite graphs.**

EXAMPLE 16 **Local Area Networks** The various computers in a building, such as minicomputers and personal computers, as well as peripheral devices such as printers and plotters, can be connected using a *local area network*. Some of these networks are based on a *star topology*, where all devices are connected to a central control device. A local area network can be represented using a complete bipartite graph $K_{1,n}$, as shown in Figure 11(a). Messages are sent from device to device through the central control device.

Links

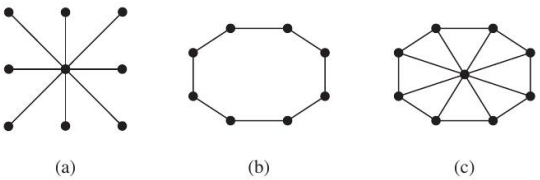


FIGURE 11 **Star, ring, and hybrid topologies for local area networks.**

EXAMPLE 1 Use adjacency lists to describe the simple graph given in Figure 1.

Solution: Table 1 lists those vertices adjacent to each of the vertices of the graph. ◀

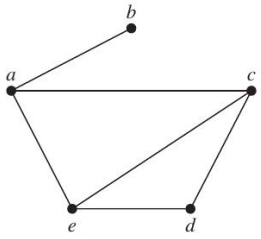


FIGURE 1 **A simple graph.**

TABLE 1 An Adjacency List for a Simple Graph.

Vertex	*Adjacent Vertices*
a	b, c, e
b	a
c	a, d, e
d	c, e
e	a, c, d

EXAMPLE 2 Represent the directed graph shown in Figure 2 by listing all the vertices that are the terminal vertices of edges starting at each vertex of the graph.

Solution: Table 2 represents the directed graph shown in Figure 2. ◀

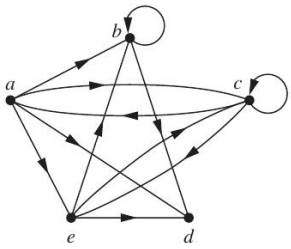


FIGURE 2 **A directed graph.**

TABLE 2 An Adjacency List for a Directed Graph.

Initial Vertex	*Terminal Vertices*
a	b, c, d, e
b	b, d
c	a, c, e
d	
e	b, c, d

10.3.3 Adjacency Matrices

Carrying out graph algorithms using the representation of graphs by lists of edges, or by adjacency lists, can be cumbersome if there are many edges in the graph. To simplify computation, graphs can be represented using matrices. Two types of matrices commonly used to represent graphs will be presented here. One is based on the adjacency of vertices, and the other is based on incidence of vertices and edges.

Suppose that $G = (V, E)$ is a simple graph where $|V| = n$. Suppose that the vertices of G are listed arbitrarily as $v_1, v_2, \ldots, v_n$. The **adjacency matrix A** (or $\mathbf{A}_G$) of G, with respect to this listing of the vertices, is the n x n zero–one matrix with 1 as its (i, j)th entry when v_i and v_j are adjacent, and 0 as its (i, j)th entry when they are not adjacent. In other words, if its adjacency matrix is $\mathbf{A} = [a_{ij}]$, then

$$a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$$

EXAMPLE 3 Use an adjacency matrix to represent the graph shown in Figure 3.

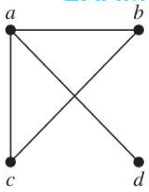


FIGURE 3
A simple graph.

Solution: We order the vertices as a, b, c, d. The matrix representing this graph is

$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}.$$

EXAMPLE 4 Draw a graph with the adjacency matrix

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

with respect to the ordering of vertices a, b, c, d.

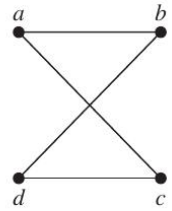
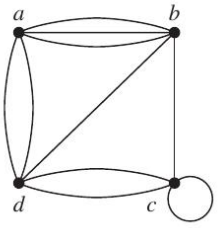


FIGURE 4

Solution: A graph with this adjacency matrix is shown in Figure 4.

EXAMPLE 5 Use an adjacency matrix to represent the pseudograph shown in Figure 5.



Solution: The adjacency matrix using the ordering of vertices a, b, c, d is

$$\begin{bmatrix} 0 & 3 & 0 & 2 \\ 3 & 0 & 1 & 1 \\ 0 & 1 & 1 & 2 \\ 2 & 1 & 2 & 0 \end{bmatrix}.$$

We used zero–one matrices in Chapter 9 to represent directed graphs. The matrix for a directed graph $G = (V, E)$ has a 1 in its (i, j)th position if there is an edge from v_i to v_j, where $v_1, v_2, \ldots, v_n$ is an arbitrary listing of the vertices of the directed graph. In other words, if $\mathbf{A} = [a_{ij}]$ is the adjacency matrix for the directed graph with respect to this listing of the vertices, then

$$a_{ij} = \begin{cases} 1 & \text{if } (v_i, v_j) \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$$

The adjacency matrix for a directed graph does not have to be symmetric, because there may not be an edge from v_j to v_i when there is an edge from v_i to v_j.

Adjacency matrices can also be used to represent directed multigraphs. Again, such matrices are not zero–one matrices when there are multiple edges in the same direction connecting two vertices. In the adjacency matrix for a directed multigraph, a_{ij} equals the number of edges that are associated to (v_i, v_j).

TRADE-OFFS BETWEEN ADJACENCY LISTS AND ADJACENCY MATRICES When a simple graph contains relatively few edges, that is, when it is **sparse**, it is usually preferable to use adjacency lists rather than an adjacency matrix to represent the graph. For example, if each vertex has degree not exceeding c, where c is a constant much smaller than n, then each adjacency list contains c or fewer vertices. Hence, there are no more than cn items in all these adjacency lists. On the other hand, the adjacency matrix for the graph has n^2 entries. Note, however, that the adjacency matrix of a sparse graph is a **sparse matrix**, that is, a matrix with few nonzero entries, and there are special techniques for representing, and computing with, sparse matrices.

Now suppose that a simple graph is **dense**, that is, suppose that it contains many edges, such as a graph that contains more than half of all possible edges. In this case, using an adjacency matrix to represent the graph is usually preferable over using adjacency lists. To see why, we compare the complexity of determining whether the possible edge $\{v_i, v_j\}$ is present. Using an adjacency matrix, we can determine whether this edge is present by examining the (i, j)th entry in the matrix. This entry is 1 if the graph contains this edge and is 0 otherwise. Consequently, we need make only one comparison, namely, comparing this entry with 0, to determine whether this edge is present. On the other hand, when we use adjacency lists to represent the graph, we need to search the list of vertices adjacent to either v_i or v_j to determine whether this edge is present. This can require $\Theta(|V|)$ comparisons when many edges are present.

10.3.4 Incidence Matrices

Another common way to represent graphs is to use **incidence matrices**. Let $G = (V, E)$ be an undirected graph. Suppose that $v_1, v_2, \ldots, v_n$ are the vertices and $e_1, e_2, \ldots, e_m$ are the edges of G. Then the incidence matrix with respect to this ordering of V and E is the $n \times m$ matrix $\mathbf{M} = [m_{ij}]$, where

$$m_{ij} = \begin{cases} 1 & \text{when edge } e_j \text{ is incident with } v_i, \\ 0 & \text{otherwise.} \end{cases}$$

EXAMPLE 6 Represent the graph shown in Figure 6 with an incidence matrix.

Solution: The incidence matrix is

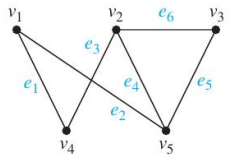


FIGURE 6 **An undirected graph.**

$$\begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{array} \begin{array}{c} \begin{array}{cccccc} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 \end{array} \\ \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \end{bmatrix} \end{array}.$$

◀

Incidence matrices can also be used to represent multiple edges and loops. Multiple edges are represented in the incidence matrix using columns with identical entries, because these edges are incident with the same pair of vertices. Loops are represented using a column with exactly one entry equal to 1, corresponding to the vertex that is incident with this loop.

EXAMPLE 7 Represent the pseudograph shown in Figure 7 using an incidence matrix.

Solution: The incidence matrix for this graph is

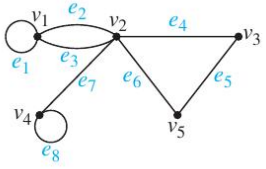


FIGURE 7
A pseudograph.

$$\begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{array} \begin{array}{c} \begin{array}{cccccccc} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 & e_7 & e_8 \end{array} \\ \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \end{array}.$$

10.3.5 Isomorphism of Graphs

We often need to know whether it is possible to draw two graphs in the same way. That is, do the graphs have the same structure when we ignore the identities of their vertices? For instance, in chemistry, graphs are used to model chemical compounds (in a way we will describe later). Different compounds can have the same molecular formula but can differ in structure. Such compounds can be represented by graphs that cannot be drawn in the same way. The graphs representing previously known compounds can be used to determine whether a supposedly new compound has been studied before.

Definition 1 The simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are *isomorphic* if there exists a one-to-one and onto function f from V_1 to V_2 with the property that a and b are adjacent in G_1 if and only if $f(a)$ and $f(b)$ are adjacent in G_2, for all a and b in V_1. Such a function f is called an *isomorphism*.* Two simple graphs that are not isomorphic are called *nonisomorphic*.

In other words, when two simple graphs are isomorphic, there is a one-to-one correspondence between vertices of the two graphs that preserves the adjacency relationship. Isomorphism of simple graphs is an equivalence relation. (We leave the verification of this as Exercise 49.)

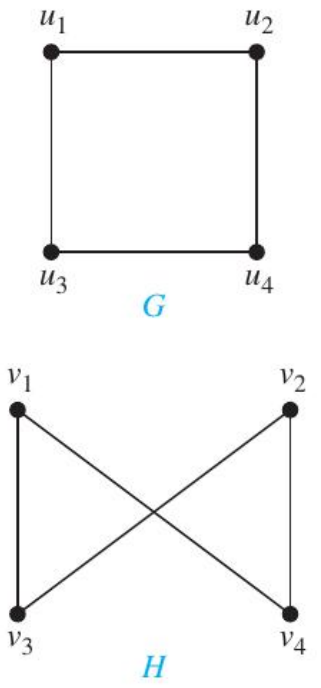


FIGURE 8 The graphs G and H.

Solution: The function f with $f(u_1) = v_1, f(u_2) = v_4, f(u_3) = v_3$, and $f(u_4) = v_2$ is a one-to-one correspondence between V and W. To see that this correspondence preserves adjacency, note that adjacent vertices in G are u_1 and u_2, u_1 and u_3, u_2 and u_4, and u_3 and u_4, and each of the pairs $f(u_1) = v_1$ and $f(u_2) = v_4, f(u_1) = v_1$ and $f(u_3) = v_3, f(u_2) = v_4$ and $f(u_4) = v_2$, and $f(u_3) = v_3$ and $f(u_4) = v_2$ consists of two adjacent vertices in H. ◀

10.3.6 Determining whether Two Simple Graphs are Isomorphic

It is often difficult to determine whether two simple graphs are isomorphic. There are $n!$ possible one-to-one correspondences between the vertex sets of two simple graphs with n vertices. Testing each such correspondence to see whether it preserves adjacency and nonadjacency is impractical if n is at all large.

Sometimes it is not hard to show that two graphs are not isomorphic. In particular, we can show that two graphs are not isomorphic if we can find a property only one of the two graphs has, but that is preserved by isomorphism. A property preserved by isomorphism of graphs is called a **graph invariant**. For instance, isomorphic simple graphs must have the same number of vertices, because there is a one-to-one correspondence between the sets of vertices of the graphs.

Isomorphic simple graphs also must have the same number of edges, because the one-to-one correspondence between vertices establishes a one-to-one correspondence between edges. In addition, the degrees of the vertices in isomorphic simple graphs must be the same. That is, a vertex v of degree d in G must correspond to a vertex $f(v)$ of degree d in H, because a vertex w in G is adjacent to v if and only if $f(v)$ and $f(w)$ are adjacent in H.

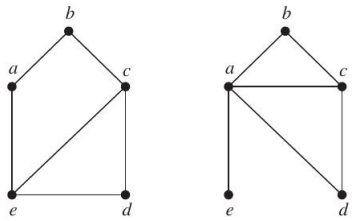


FIGURE 9 **The graphs G and H.**

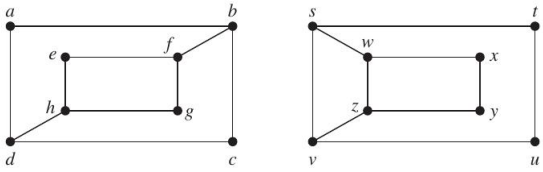


FIGURE 10 **The graphs G and H.**

EXAMPLE 9 Show that the graphs displayed in Figure 9 are not isomorphic.

Extra Examples ➤ *Solution:* Both G and H have five vertices and six edges. However, H has a vertex of degree one, namely, e, whereas G has no vertices of degree one. It follows that G and H are not isomorphic.

Solution: The graphs G and H both have eight vertices and 10 edges. They also both have four vertices of degree two and four of degree three. Because these invariants all agree, it is still conceivable that these graphs are isomorphic.

However, G and H are not isomorphic. To see this, note that because $\deg(a) = 2$ in G, a must correspond to either t, u, x, or y in H, because these are the vertices of degree two in H. However, each of these four vertices in H is adjacent to another vertex of degree two in H, which is not true for a in G.

To show that a function f from the vertex set of a graph G to the vertex set of a graph H is an isomorphism, we need to show that f preserves the presence and absence of edges. One helpful way to do this is to use adjacency matrices. In particular, to show that f is an isomorphism, we can show that the adjacency matrix of G is the same as the adjacency matrix of H, when rows and columns are labeled to correspond to the images under f of the vertices in G that are the labels of these rows and columns in the adjacency matrix of G. We illustrate how this is done in Example 11.

Determine whether the graphs G and H displayed in Figure 12 are isomorphic.

Solution: Both G and H have six vertices and seven edges. Both have four vertices of degree two and two vertices of degree three. It is also easy to see that the subgraphs of G and H consisting of all vertices of degree two and the edges connecting them are isomorphic (as the reader should verify). Because G and H agree with respect to these invariants, it is reasonable to try to find an isomorphism f.

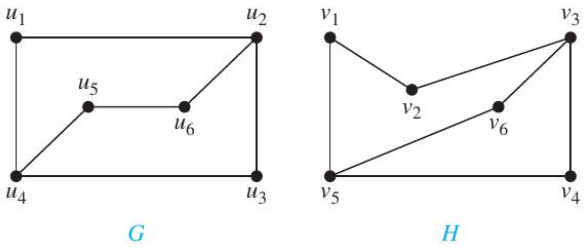


FIGURE 12 **Graphs G and H.**

We now will define a function f and then determine whether it is an isomorphism. Because $\deg(u_1) = 2$ and because u_1 is not adjacent to any other vertex of degree two, the image of u_1 must be either v_4 or v_6, the only vertices of degree two in H not adjacent to a vertex of degree two. We arbitrarily set $f(u_1) = v_6$. [If we found that this choice did not lead to isomorphism, we would then try $f(u_1) = v_4$.] Because u_2 is adjacent to u_1, the possible images of u_2 are v_3 and v_5. We arbitrarily set $f(u_2) = v_3$. Continuing in this way, using adjacency of vertices and degrees as a guide, we set $f(u_3) = v_4$, $f(u_4) = v_5$, $f(u_5) = v_1$, and $f(u_6) = v_2$. We now have a one-to-one correspondence between the vertex set of G and the vertex set of H, namely, $f(u_1) = v_6$, $f(u_2) = v_3, f(u_3) = v_4, f(u_4) = v_5, f(u_5) = v_1, f(u_6) = v_2$. To see whether f preserves edges, we examine the adjacency matrix of G,

$$\mathbf{A}_G = \begin{array}{c} \\ u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \end{array} \begin{array}{c} \begin{array}{cccccc} u_1 & u_2 & u_3 & u_4 & u_5 & u_6 \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix} \end{array},$$

and the adjacency matrix of H with the rows and columns labeled by the images of the corresponding vertices in G,

$$\mathbf{A}_H = \begin{array}{c} \\ v_6 \\ v_3 \\ v_4 \\ v_5 \\ v_1 \\ v_2 \end{array} \begin{array}{c} \begin{array}{cccccc} v_6 & v_3 & v_4 & v_5 & v_1 & v_2 \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix} \end{array}.$$

Because $\mathbf{A}_G = \mathbf{A}_H$, it follows that f preserves edges. We conclude that f is an isomorphism, so G and H are isomorphic. Note that if f turned out not to be an isomorphism, we would *not* have established that G and H are not isomorphic, because another correspondence of the vertices in G and H may be an isomorphism. ◀

Tutorial Praktikum

Tutorial01.py

Task Praktikum

Pada praktikum matdis lanjut mahasiswa boleh menggunakan tools AI, dokumentasi official dan lain-lain, sesuai RPS tidak semua tugas akan dinilai, namun akan digunakan sebagai landasan untuk mengerjakan pekerjaan selanjutnya / modul selanjutnya.

Task Individu

Task01_Individu.py

1. Buatlah sebuah graf Complete K_4, graf Cycle C_5, dan graf wheel W_6. Visualisasikan ketiga graf tersebut.
2. Untuk setiap graf, cetak properti berikut: jumlah simpul, jumlah sisi, dan derajat setiap simpul.

Tuliskan Lesson learnt task ini (boleh screenshoot hasil akhir)

Task02_Individu.py

1. Analisis graf: hitung derajat setiap simpul dan verifikasi `Handshaking Theorem`.
2. Representasikan graf ke dalam **Adjacency List** dan **Adjacency Matrix**.

Tuliskan Lesson learnt task ini (boleh screenshoot hasil akhir)

Task Kelompok (Soal Kolaboratif & Sharing)

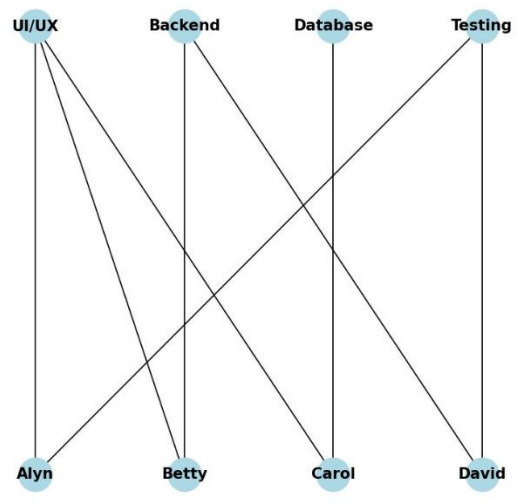
Setiap anggota kelompok akan memilih satu dari tiga studi kasus (soal 4, 5, atau 6) untuk dikerjakan. Setelah selesai, setiap mahasiswa akan mempresentasikan hasil analisisnya kepada kelompok.

Setelah semua anggota selesai mengerjakan, setiap mahasiswa harus:

1. Menjelaskan hasil analisis dan kode programnya kepada anggota kelompok lain.
2. Membuat lesson learnt tentang apa yang dipelajari dari studi kasus teman-teman mereka.

Task03_Timproyek.py

- **Skenario**: Anda adalah manajer proyek dengan 4 `developer` (Alyn, Betty, Carol, David) dan 4 `task` (UI/UX, Backend, Database, Testing).
- **Keempat `developer` memiliki keahlian sebagai berikut**: Alyn (UI/UX, Testing), Betty (UI/UX, Backend), Carol (UI/UX, Database), David (Backend, Testing).
- **Tugas**:
    - Modelkan masalah ini sebagai **Bipartite Graph**.
    - Representasikan graf ke dalam **Adjacency List** dan **Adjacency Matrix**.
    - Analisis graf: hitung derajat setiap simpul dan verifikasi `Handshaking Theorem`.
    - Identifikasi apakah ada **isomorfisme** antara graf Anda dengan graf khusus yang sudah dipelajari.




```
>>> %Run Task03_Timproyek.py

Graph with 8 nodes and 8 edges
[('Alyn', 'UI/UX'), ('Alyn', 'Testing'), ('Betty', 'UI/UX'), ('Betty', 'Backend'), ('
Carol', 'UI/UX'), ('Carol', 'Database'), ('David', 'Backend'), ('David', 'Testing')]

Adjacency List:
{'Alyn': ['UI/UX', 'Testing'], 'Betty': ['UI/UX', 'Backend'], 'Carol': ['UI/UX', 'Dat
abase'], 'David': ['Backend', 'Testing'], 'UI/UX': ['Alyn', 'Betty', 'Carol'], 'Backe
nd': ['Betty', 'David'], 'Database': ['Carol'], 'Testing': ['Alyn', 'David']}

Adjacency Matrix:
[[0. 0. 0. 0. 0. 1. 1.]
 [0. 0. 1. 0. 0. 1. 0.]
 [0. 1. 0. 0. 0. 0. 1.]
 [0. 0. 0. 0. 1. 0. 0. 1.]
 [0. 0. 0. 1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0. 1. 0.]
 [1. 0. 0. 0. 0. 1. 0. 0.]
 [1. 0. 1. 1. 0. 0. 0. 0.]]

Derajat Tiap Node
  - Nodes 'Alyn': 2
  - Nodes 'Betty': 2
  - Nodes 'Carol': 2
  - Nodes 'David': 2
  - Nodes 'UI/UX': 3
  - Nodes 'Backend': 2
  - Nodes 'Database': 1
  - Nodes 'Testing': 2
Total derajat semua simpul = 16
2 * Jumlah Sisi (2 * 8) = 16
Status: TERVERIFIKASI (Handshaking Theorem terbukti benar)
Isomorfik dengan Graf Lengkap (K8)?: False
Isomorfik dengan Graf Lingkaran (C8)?: False
Isomorfik dengan Graf Lintasan (P8)?: False
Isomorfik dengan Graf Bipartit Lengkap (K4,4)?: False
Isomorfik dengan Graf Roda (W7)?: False
Isomorfik dengan Graf Kubus (Q3)?: False
Isomorfik dengan Graf Bintang (S7)?: False

[KESIMPULAN]
Graf G1 TIDAK ISOMORFIK dengan graf khusus standar mana pun.
Graf G1 adalah Graf Bipartit biasa (subgraf dari K4,4), bukan Graf Bipartit Lengkap.

>>>
```

Lesson learnt:

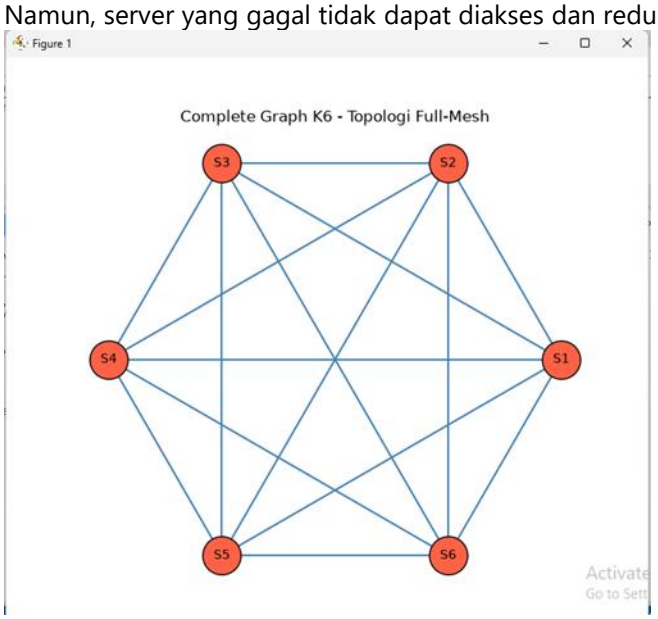
- Memahami konsep Bipartite Graph untuk memodelkan hubungan antara developer dan task berdasarkan keahlian yang dimiliki.
- Mencoba merepresentasikan graf menggunakan Adjacency List dan Adjacency Matrix dengan bantuan library NetworkX dan NumPy pada Python.
- Memahami konsep derajat simpul (degree) untuk mengetahui jumlah hubungan yang dimiliki setiap developer maupun task, sehingga dapat mengidentifikasi task

yang memiliki banyak atau sedikit kandidat pengerjaan.
- Memverifikasi Handshaking Theorem, yaitu jumlah seluruh derajat simpul sama dengan dua kali jumlah sisi pada graf, sebagai bentuk pemeriksaan konsistensi struktur graf.
- Memahami konsep isomorfisme graf dengan membandingkan struktur graf yang terbentuk dengan beberapa graf khusus, seperti graf lengkap, graf lingkaran, graf lintasan, dan graf bipartit lengkap.
- Melatih kemampuan memodelkan permasalahan pembagian tugas ke dalam struktur graf untuk membantu menganalisis hubungan antara sumber daya dan kebutuhan pekerjaan dalam suatu proyek.

Task04_FullMesh.py

- **Skenario**: Anda adalah administrator jaringan yang menganalisis topologi full-mesh dengan enam server yang terhubung langsung satu sama lain.
- **Tugas**:
  - Modelkan ini sebagai Complete Graph (K_n) dan gambarkan.
  - Representasikan graf ini dalam Incidence Matrix.
  - Analisis graf: hitung derajat setiap simpul dan verifikasi `Handshaking Theorem`.
  - Jelaskan pengaruh kegagalan satu server pada connectivity graf.

Pengaruh kegagalan satu server:
Jika S1 gagal, 5 koneksi ikut hilang.
Graf tersisa adalah K5 dengan 10 koneksi.
Derajat setiap server yang tersisa: [4, 4, 4, 4, 4]
Verifikasi BFS -- graf tersisa terhubung penuh: True
Namun, server yang gagal tidak dapat diakses dan redundansi total berkurang.



```
=== COMPLETE GRAPH K6 / FULL-MESH ===
Simpul (server): ['S1', 'S2', 'S3', 'S4', 'S5', 'S6']
Jumlah sisi/koneksi: |E| = 15 = 6(6 - 1)/2
Daftar koneksi:
S1-S2, S1-S3, S1-S4, S1-S5, S1-S6, S2-S3, S2-S4, S2-S5, S2-S6, S3-S4, S3-S5, S3-S6, S4-S5, S4-S6, S5-S6

=== MATRIKS INSIDENSI ===
Baris = simpul, kolom = sisi
    e01 e02 e03 e04 e05 e06 e07 e08 e09 e10 e11 e12 e13 e14 e15
S1    1   1   1   1   1   0   0   0   0   0   0   0   0   0   0
S2    1   0   0   0   0   1   1   1   1   0   0   0   0   0   0
S3    0   1   0   0   0   1   0   0   0   1   1   1   0   0   0
S4    0   0   1   0   0   0   1   0   0   1   0   0   1   1   0
S5    0   0   0   1   0   0   0   1   0   0   1   0   1   0   1
S6    0   0   0   0   1   0   0   0   1   0   0   1   0   1   1

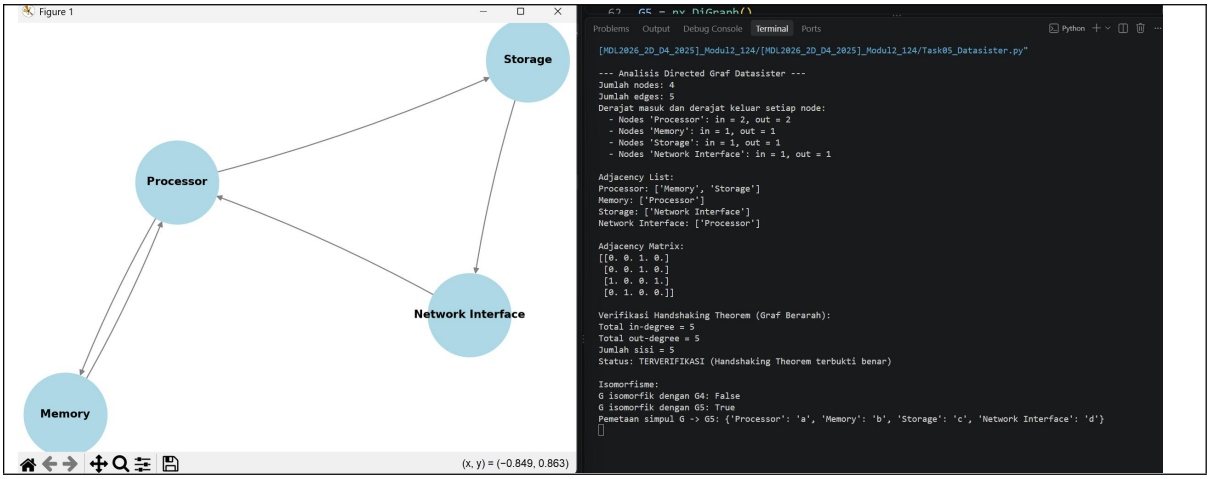
=== ANALISIS DERAJAT ===
deg(S1) = 5
deg(S2) = 5
deg(S3) = 5
deg(S4) = 5
deg(S5) = 5
deg(S6) = 5
Total derajat = 30
2|E| = 2 * 15 = 30
Handshaking Theorem terpenuhi: True
```

Lesson Learnt:
Praktikum pemodelan K6 ini membuktikan tingginya keandalan topologi *full-mesh*, di mana kegagalan satu simpul hanya mereduksi graf menjadi K5 tanpa mengorbankan konektivitas jaringan secara keseluruhan. Simulasi ini juga secara presisi memverifikasi *Handshaking Theorem* dan menunjukkan bahwa representasi *Incidence Matrix* efektif memetakan koneksi namun memakan banyak memori pada graf yang padat. Secara praktis, pertumbuhan jumlah sisi eksponensial sebesar n(n – 1)/2 menyadarkan saya bahwa topologi ini memiliki tantangan skalabilitas dan biaya infrastruktur yang sangat tinggi jika diterapkan pada jaringan berskala besar.

Task05_Datasister.py

- **Skenario**: Anda menganalisis aliran data unidirectional antar 4 komponen: Processor, Memory, Storage, Network Interface.
- **Aliran data:** Processor -> Memory, Processor -> Storage, Memory -> Processor, Storage -> Network Interface, Network Interface -> Processor.
- **Tugas**:
    - Modelkan ini sebagai Directed Graph.
    - Representasikan graf ke Adjacency List dan Adjacency Matrix.
    - Analisis graf: hitung in-degree dan out-degree setiap simpul dan verifikasi Handshaking Theorem versi berarah.
    - Identifikasi isomorfisme dengan graf sederhana lain.

Pada task ini dibuat graf dengan 4 simpul dan 5 sisi berarah sesuai skenario dan aliran data yang sudah ditentukan. Dengan adanya arah pada graf ini, arah sisi pada model menghasilkan adjacency matriks yang asimetris dan adjacency list yang hanya mencatat simpul tujuan. Terkait Handshaking Theorem versi berarah, total in-degree dan out-degree harus selalu sama dengan jumlah sisi. Kemudian, dengan membandingkan derajat simpul didapatkan isomorfisme antargraf.


Figure 1
Storage
Processor
Network Interface
Memory
(x, y) = (−0.849, 0.863)
Problems
Output
Debug Console
Terminal
Ports
Python
[MDL2026_2D_D4_2025]_Modul2_124/[MDL2026_2D_D4_2025]_Modul2_124/Task05_Datasister.py"
--- Analisis Directed Graf Datasister ---
Jumlah nodes: 4
Jumlah edges: 5
Derajat masuk dan derajat keluar setiap node:
- Nodes 'Processor': in = 2, out = 2
- Nodes 'Memory': in = 1, out = 1
- Nodes 'Storage': in = 1, out = 1
- Nodes 'Network Interface': in = 1, out = 1
Adjacency List:
Processor: ['Memory', 'Storage']
Memory: ['Processor']
Storage: ['Network Interface']
Network Interface: ['Processor']
Adjacency Matrix:
[[0. 0. 1. 0.]
[0. 0. 1. 0.]
[1. 0. 0. 1.]
[0. 1. 0. 0.]]
Verifikasi Handshaking Theorem (Graf Berarah):
Total in-degree = 5
Total out-degree = 5
Jumlah sisi = 5
Status: TERVERIFIKASI (Handshaking Theorem terbukti benar)
Isomorfisme:
G isomorfik dengan G4: False
G isomorfik dengan G5: True
Pemetaan simpul G -> G5: {'Processor': 'a', 'Memory': 'b', 'Storage': 'c', 'Network Interface': 'd'}

Use the "Insert Citation" button to add citations to this document.

https://networkx.org/documentation/stable/reference/generated/networkx.generators.classic.complete_graph.html

https://networkx.org/documentation/stable/reference/algorithms/cycles.html

https://networkx.org/documentation/stable/reference/generated/networkx.generators.classic.wheel_graph.html

https://networkx.org/documentation/stable/reference/algorithms/bipartite.html

https://networkx.org/documentation/stable/reference/classes/digraph.html

https://www.google.com/search?client=firefox-b-d&q=networkx+star+graph

https://networkx.org/documentation/stable/reference/generators.html